

CO3_AT1:ANALYTICAL BASED QUESTION

8/5/26, 11:20 AM

Student Submissions — Application Based Questions

SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

Student 34 **Vaibhava Lakshmi K** 192511310RD

2 / 2 submitted

Q1. Student Grade Book Application

CODE

```
students = {}

for i in range(5):
    name = input()
    marks = list(map(int, input().split()))
    students[name] = marks

result = {}

for name, marks in students.items():
    total = sum(marks)
    avg = total / 5

    if avg >= 90:
        grade = "O"
    elif avg >= 80:
        grade = "A+"
    elif avg >= 70:
        grade = "A"
    elif avg >= 60:
        grade = "B+"
    elif avg >= 50:
        grade = "B"
    else:
        grade = "F"

    result[name] = (total, avg, grade)

topper = max(result, key=lambda x: result[x][1])
print("Topper:", topper)

subject_avg = []

for i in range(5):
    s = 0
    for m in students.values():
        s += m[i]
    subject_avg.append(s / 5)

print("Highest Mean Subject:", subject_avg.index(max(subject_avg)) + 1)

for name, data in sorted(result.items(), key=lambda x: x[1][1], reverse=True):
    print(name, data[0], data[1], data[2])
```

```
print("Highest Mean Subject:", subject_avg.index(max(subject_avg)) + 1)

for name, data in sorted(result.items(), key=lambda x: x[1][1], reverse=True):
    print(name, data[0], data[1], data[2])
```

INPUT

```
Alice
90 85 80 95 88
Bob
70 75 80 65 60
Charlie
95 92 90 94 96
David
60 55 58 62 65
Eva
85 80 82 84 83
```

OUTPUT

(no output)

8/5/26, 11:20 AM

Student Submissions — Application Based Questions

SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions F

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {
        'name': name,
        'balance': balance,
        'txns': []
    }

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(("Deposit", amount))

def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] >= amount:
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(("Withdraw", amount))
        else:
            print("Insufficient balance")

def mini_statement(acc_no):
    if acc_no in accounts:
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)

create_account(1001, "Ravi", 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)

mini_statement(1001)

print("Balance:", accounts[1001]['balance'])
```

OUTPUT

(no output)